

C++ Programming: Problem Solving and Programming

Chapter 10 Bitwise Operator

Objectives

- Understanding of the concepts
- Identify and the use of bitwise operators
- Application of the operators in problem solving
- Optimize and compare with the arithmetic or logical operators
- Real world applications

Introduction

- The integer types, such as `int` and `long`, are machine dependent.
- In one computer, the size of `int` may be four bytes, in another computer it may be two bytes.
- While many bitwise applications work well on machine-dependent integer types, other applications need to assure that the size is fixed.
- C++ allows us to define integer types of sizes 8, 16, 32, and 64 bits.

Introduction

- Bitwise operators ($\sim$, $\&$, $|$, $\wedge$, $\ll$, $\gg$) take operands (number) of any integer type but treat an operand as a collection of bits rather than as a single number.

e.g.

2	□	11011011
5	□	01001110

 Collection of bits

Bitwise Negation ‘~’

- Application of the negation “~” operator to an integer produces a value in which each bit of the operand has been replaced by its negation.

i.e. each 0 is replaced by 1 and vice versa

- Negation (One’s Complement) Truth Table

Original Bit	Result
0	1
1	0

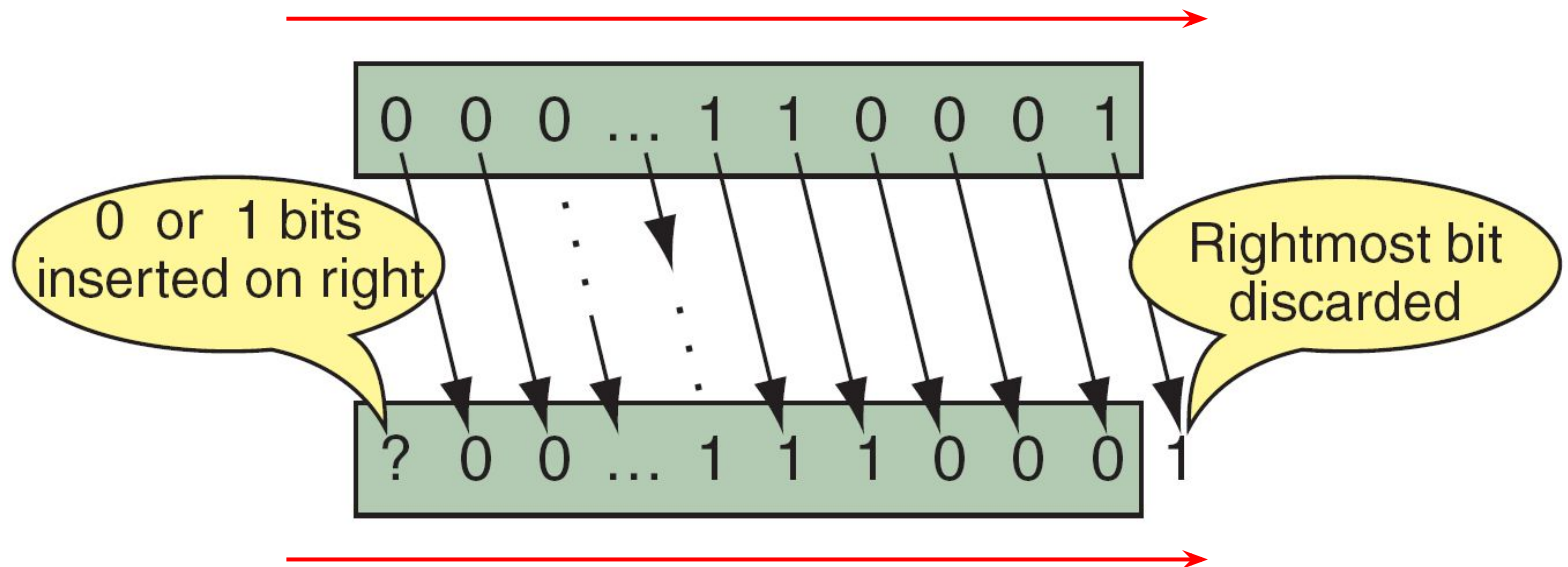
Bitwise Negation ‘~’

Example:

A	1001	0110	0001	0001
~A	0110	1001	1110	1110

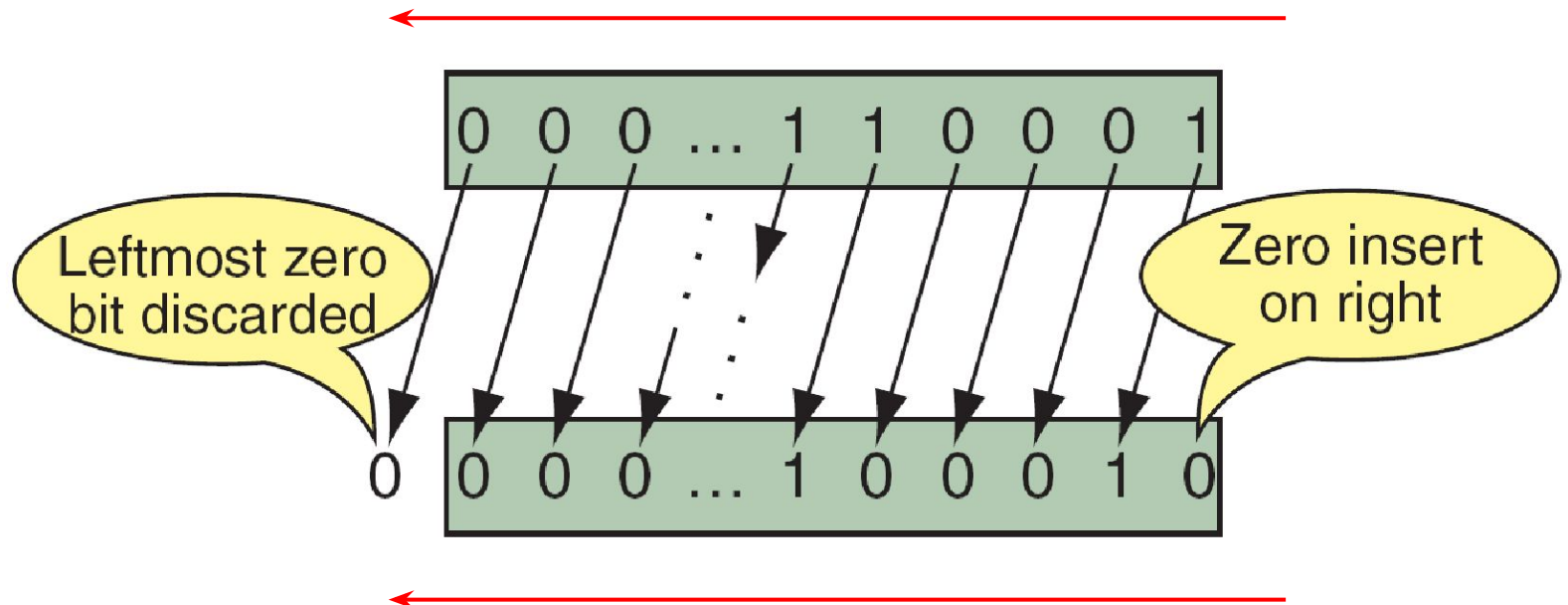
Shift Operator ‘<<’ & ‘>>’

- The left “<<” and right “>>” shift operators take an operand and return its value shifted left or right by a specified number of bits.
- Right Shift Operator (>>)



Shift Operator '<<' & '>>'

- Left Shift Operator (<<)



Shift Operator '<<' & '>>'

- Example :

A 1001 0110 0001 0001

A<<1

Left

~~1~~001 0110 0001 0001~~0~~
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
0010 1100 0010 0010

A>>3

Right

000~~1~~001 0110 0001 000~~1~~
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
0001 0010 1100 0010

Bitwise Logical Operators

- There are three (3) bitwise logical operators.

exclusively-OR

i.e: AND (&), OR (|) and XOR (^)

These operators take two operands (2 numbers).

- AND(&) Truth Table

First Operand Bit	Second Operand Bit	Result
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise Logical Operators

- OR(|) Truth Table

First Bit	Second Bit	Result
0	0	0
0	1	1
1	0	1
1	1	1

- XOR(^) Truth Table

First Bit	Second Bit	Result
0	0	0
0	1	1
1	0	1
1	1	0

Bitwise Logical Operators

- Example :

A	1001	0110	0001	0001
B	0001	1010	1101	1001

A&B	0001	0010	0001	0001
A B	1001	1110	1101	1001
A^B	1000	1100	1100	1000

Exercise 1

The value of **M** and **N** in binary format is **0011 1010** and **1010 1101** respectively. Determine the result of the following expressions:

$\sim M$ **1.**

$M \mid N$ **2.**

$M \wedge N$ **3.**

$\sim(N \wedge M)$ **4.**

$N \& M$ **5.**

$N \& \sim M$ **6.**

Exercise 2

The value of **M** in binary format is **1011 0110**.
Determine the result of the following expressions:

M >> 2 1.

M << 3 2.

M >> 2 << 3 3.

M << 1 >> 4 4.

Applications

1. MASKING

- In many programs, bits are used as binary flags: 0 is off, and 1 is on.
- To set and test the flags, we use a bit **mask**.
- A mask is a variable or constant, usually stored in a byte or short integer.
- The bits are numbered from the least significant bit (rightmost), starting at 0.

Applications

2. DATA ENCRYPTION

- Encrypting a file makes its contents unreadable while preserving the information it contains.
- One of the simplest ways to encrypt text is to exclusively-OR (XOR) the text with a key.
- To decrypt the encrypted file; we then use the same method and the same key.

Example:

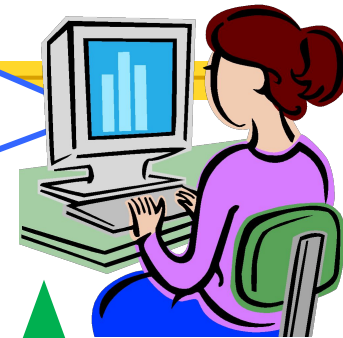
To encrypt:

Data: 0101^A 0000^B (ASCII)

KEY: 0001 0010

XOR operator 0100 0010 (Encrypted data)

Transmit to
server computer



Password
accepted

To decrypt:

Data during transmission: 0100 0010

^KEY: 0001 0010

Original Data
(Decrypted data)

0101 0000

A B

